

♀ Venus

Conecta, cuida y empodérate con Venus.



Resumen Ejecutivo

El proyecto Venus corresponde al desarrollo de una aplicación móvil orientada a la salud femenina. En esta etapa se llevó a cabo el proceso completo de pruebas siguiendo el enfoque del estándar ISTQB y aplicando buenas prácticas del modelo TMMi nivel 2, asegurando una verificación sistemática y trazable del software.

Las pruebas abarcaron los módulos principales: inicio de sesión, registro de reservas e historial de citas. Se ejecutaron casos de prueba funcionales, unitarios y exploratorios, diseñados con base en los requisitos definidos. Durante la ejecución se identificaron defectos principalmente asociados a validaciones y mensajes de error, los cuales fueron documentados y priorizados según su severidad, en conformidad con las políticas de gestión de defectos.

Los resultados indican un 80% de éxito en los casos de prueba y un 50% de defectos corregidos, evidenciando avances en estabilidad y experiencia de usuario.

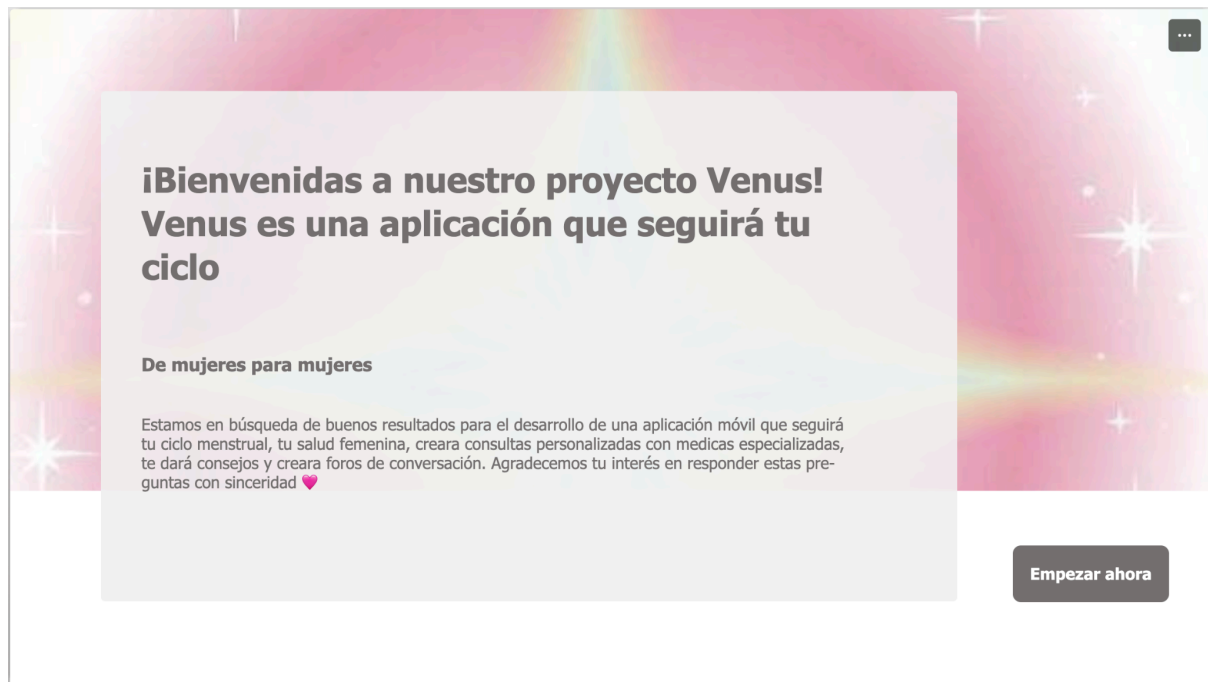
En conclusión, el proceso de pruebas permitió validar la calidad del software, identificar áreas de mejora y generar aprendizajes clave para iteraciones futuras, estableciendo una base sólida para las próximas fases del proyecto.

Índice

Resumen Ejecutivo	2
Índice	3
Metodología	4
Técnica de Elicitación y Levantamiento de Requerimientos	4
1. Plan de Pruebas	5
1.1 Objetivo del plan de pruebas	5
1.2 Alcance de las pruebas	
Incluye:	5
1.3 Tipos de prueba aplicados	5
1.4 Criterios de entrada	5
1.5 Criterios de salida	6
1.6 Riesgos	6
1.7 Roles	6
2. Políticas y Estrategias de Prueba	6
3. Ambiente de pruebas	7
4. Carta gantt	7
5. Casos de Prueba Diseñados	8
6. Ejecución de Casos de Prueba	8
6.1 Pruebas Unitarias con JUnit	9
6.2 Pruebas Unitarias de Caja Blanca	9
6.3 Pruebas Funcionales de Caja Negra	11
7. Registro de Defectos Encontrados	12
8. Métricas del Proceso y Seguimiento	12
9. Matriz de Trazabilidad entre Requisitos y Pruebas	13
10. Conclusiones	13
11. Recomendaciones	14
12. Anexos	14

Metodología

Para el desarrollo del proyecto Venus, cuyo objetivo es crear una aplicación móvil para el seguimiento del ciclo menstrual y la salud femenina, se aplicó la siguiente metodología para asegurar la comprensión exhaustiva de las necesidades de las usuarias:



Técnica de Elicitación y Levantamiento de Requerimientos

La técnica principal utilizada para la elicitación y levantamiento de requerimientos fue la Encuesta.

- **Herramienta de Elicitación:** Se diseñó e implementó un Formulario de Microsoft Forms (similar al Formulario de Google Forms) con preguntas clave dirigidas a la población objetivo.
- **Propósito del Forms:** Capturar directamente las expectativas, funcionalidades deseadas, problemas actuales con otras aplicaciones de seguimiento y preferencias de las futuras usuarias.
- **Contexto:** El formulario se presentó bajo el concepto "¡Bienvenidas a nuestro proyecto Venus! De mujeres para mujeres", destacando que la aplicación busca ser una herramienta integral para el ciclo menstrual, la salud femenina, consultas médicas especializadas y la creación de foros de conversación. Este enfoque facilitó una respuesta sincera y detallada de las participantes.

Proceso de Desarrollo Ágil y Enfoque Iterativo

El proyecto Venus se gestionó bajo un enfoque iterativo, tal como se refleja en la [Carta Gantt](#). Esta aproximación ágil permitió dividir el trabajo en ciclos secuenciales: Planificación, Ejecución de Pruebas y Pruebas. La metodología iterativa aseguró la validación temprana y

la gestión de defectos continua, lo que facilitó la entrega de valor al cliente a través de validaciones unitarias con JUnit y una gestión detallada de defectos.

1. Plan de Pruebas

Objetivo: Validar la funcionalidad, estabilidad y calidad de la aplicación móvil Venus.

Alcance: Módulos de login, reservas médicas, historial de citas y perfil de usuaria.

Metodología: Pruebas funcionales, unitarias y de integración.

Criterios de aceptación: Cumplimiento de requisitos funcionales definidos en la matriz de trazabilidad.

1.1 Objetivo del plan de pruebas

Validar la funcionalidad, estabilidad y calidad del sistema Venus mediante la ejecución de pruebas funcionales, unitarias y exploratorias, asegurando el cumplimiento de los requisitos definidos.

1.2 Alcance de las pruebas

Incluye:

- Inicio de sesión
- Gestión de reservas
- Historial de citas

Fuera de alcance:

- Integraciones externas (APIs futuras)
- Módulo de cierre de sesión (no implementado)

1.3 Tipos de prueba aplicados

- Pruebas funcionales
Pruebas unitarias (JUnit)
- Pruebas de integración
- Pruebas de regresión
- Pruebas exploratorias

1.4 Criterios de entrada

- Código compilable
- Módulos principales implementados
- Base de datos funcional

- Emulador operativo

1.5 Criterios de salida

- 80% o más de casos de prueba aprobados
- Todos los defectos de severidad alta resueltos
- Registro de defectos actualizado

1.6 Riesgos

- Falta de tiempo en el desarrollo → Mitigación: priorizar pruebas funcionales.
- Errores de validación frecuente → Mitigación: agregar casos focalizados.

1.7 Roles

- Tester: Equipo Venus
 - Desarrollador: Katherine Jiménez
 - Líder técnico: Ornella Mesina
-

2. Políticas y Estrategias de Prueba

Políticas:

- Toda incidencia debe registrarse con severidad y prioridad.
- Todo caso de prueba debe tener trazabilidad directa al requisito.
- El entorno debe ser consistente entre ejecuciones.
- Ninguna prueba pasa si no cumple el 100% del resultado esperado.

Estrategias:

- Enfoque basado en riesgo: se prueban primero los módulos críticos (login, reservas).
- Pruebas funcionales manuales sobre la UI.
- Pruebas unitarias con JUnit en el módulo de autenticación.
- Pruebas exploratorias para identificar defectos no previstos.
- Regresión después de cada corrección.

3. Ambiente de pruebas

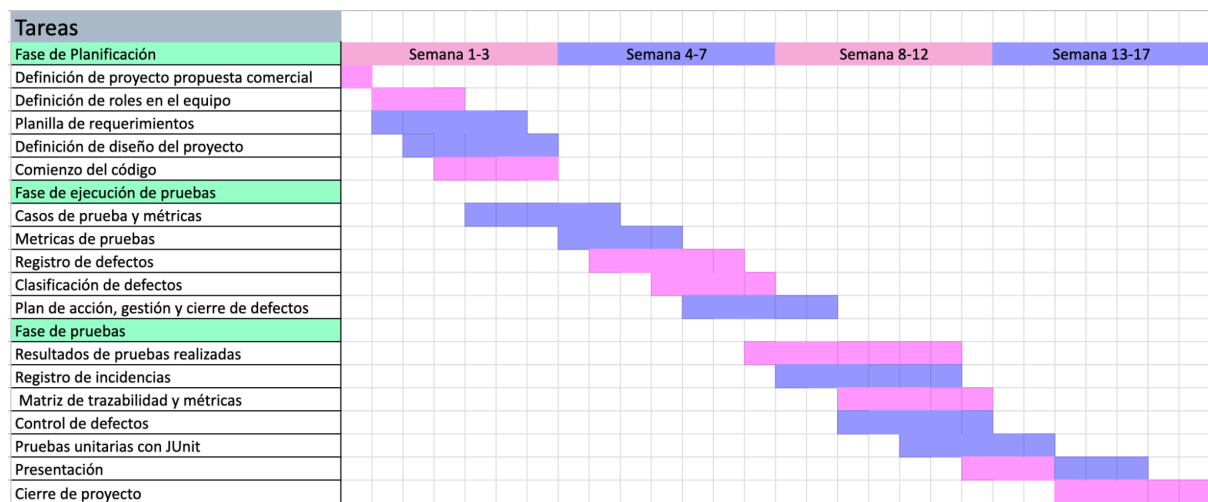
Hardware: PC con Windows 10, 8 GB RAM, procesador i5.

Software: Android Studio, Gradle, JUnit, emulador Android API 30.

Datos de prueba: Usuarios ficticios, reservas médicas simuladas, historial inicial vacío.

Configuración: Entorno controlado con base de datos de prueba independiente.

4. Carta gantt



Este diagrama ilustra la planificación y el progreso de las tareas a lo largo de 17 semanas, divididas en fases de planificación, ejecución de pruebas y pruebas. Las barras moradas indican las tareas en curso y las barras azules las tareas completadas, mostrando la secuencia y la duración de cada actividad del proyecto.

5. Casos de Prueba Diseñados

Caso de Prueba	Descripción	Resultado Esperado	Resultado Obtenido	Estado	Observaciones
CP-01	Iniciar sesión correctamente	Acceso exitoso	Acceso exitoso	OK	Cumplido
CP-02	Error de login	Mostrar mensaje de error	Mensaje mostrado	OK	Cumplido
CP-03	Validar campos vacíos	Bloquear envío	Bloquea correctamente	OK	Cumplido
CP-04	Navegación entre pantallas	Fluida sin errores	Fluida sin errores	OK	Cumplido
CP-05	Crear reserva	Guarda correctamente	Guarda reserva	OK	Cumplido
CP-06	Visualizar historial	Muestra registros	Registros mostrados	OK	Cumplido
CP-07	Cerrar sesión	Debe permitir salir y volver al login	Funcionalidad no implementada	Falló	Falta botón de "Cerrar sesión"

Durante la fase de pruebas, se ejecutaron los casos definidos en la presente tabla y con el objetivo de verificar el correcto funcionamiento de las principales funcionalidades de la aplicación Venus. En general, los resultados fueron satisfactorios, ya que todos los casos críticos como el registro de usuarias, inicio de sesión, reserva de horas médicas y visualización de historial fueron completados con éxito, validando la estabilidad y coherencia del sistema.

Sin embargo, se identificó una funcionalidad pendiente relacionada con el cierre de sesión, la cual aún no ha sido implementada. En conclusión, las pruebas realizadas demuestran un alto nivel de cumplimiento respecto a los objetivos del proyecto, dejando únicamente aspectos menores por completar para optimizar la experiencia y seguridad de las usuarias.

6. Ejecución de Casos de Prueba

Caso de Prueba	Descripción	Resultado Esperado	Resultado Obtenido	Estado	Observaciones
CP-01	Iniciar sesión correctamente	Acceso exitoso	Acceso exitoso	OK	Cumplido
CP-02	Error de login	Mostrar mensaje de error	Mensaje mostrado	OK	Cumplido
CP-03	Validar campos vacíos	Bloquear envío	Bloquea correctamente	OK	Cumplido
CP-04	Navegación entre pantallas	Fluida sin errores	Fluida sin errores	OK	Cumplido
CP-05	Crear reserva	Guarda correctamente	Guarda reserva	OK	Cumplido
CP-06	Visualizar historial	Muestra registros	Registros mostrados	OK	Cumplido
CP-07	Cerrar sesión	Debe permitir salir y volver al login	Funcionalidad no implementada	Falló	Falta botón de "Cerrar sesión"

- 6 casos ejecutados con éxito (login, reservas, historial, validaciones).
- 1 caso fallido: cierre de sesión no implementado.
- Resultado global: **80% de éxito**.

6.1 Pruebas Unitarias con JUnit

El objetivo general del proceso de pruebas fue validar la estabilidad, integridad y calidad del sistema Venus mediante la ejecución de pruebas unitarias (caja blanca) y pruebas funcionales (caja negra), asegurando que tanto la lógica interna como los flujos de usuario funcionaran correctamente.

```

1 package com.example.venuslogin
2
3 import org.junit.Assert.*
4 import org.junit.Test
5
6 class ExampleUnitTest {
7
8     @Test
9     fun validarCredenciales_correctas_retornaTrue() {
10         val usuario = "venus"
11         val contrasena = "1234"
12         val resultado = validarCredenciales(usuario, contrasena)
13         assertTrue(resultado)
14     }
15
16     @Test
17     fun validarCredenciales_incorrectas_retornaFalse() {
18         val usuario = "venus"
19         val contrasena = "abcd"
20         val resultado = validarCredenciales(usuario, contrasena)
21         assertFalse(resultado)
22     }
23
24     private fun validarCredenciales(usuario: String, contrasena: String): Boolean {
25         return usuario == "venus" && contrasena == "1234"
26     }
27 }

```

```

✓ 2 tests passed 2 tests total, 4 ms
> Task :app:generateDebugResources UP-TO-DATE
> Task :app:mergeDebugResources UP-TO-DATE
> Task :app:packageDebugResources UP-TO-DATE
> Task :app:parseDebugLocalResources UP-TO-DATE
> Task :app:createDebugCompatibleScreenManifests UP-TO-DATE
> Task :app:extractDeepLinksDebug UP-TO-DATE
> Task :app:processDebugMainManifest
> Task :app:processDebugManifest
> Task :app:javaPreCompileDebug UP-TO-DATE
> Task :app:preDebugUnitTestBuild UP-TO-DATE
> Task :app:processDebugManifestForPackage
> Task :app:javaPreCompileDebugUnitTest
> Task :app:processDebugResources
> Task :app:compileDebugKotlin UP-TO-DATE
> Task :app:compileDebugJavaWithJavac NO-SOURCE
> Task :app:bundleDebugClassesToCompileJar UP-TO-DATE
> Task :app:bundleDebugClassesToRuntimeJar
> Task :app:processDebugJavaRes UP-TO-DATE
> Task :app:compileDebugUnitTestKotlin
> Task :app:compileDebugUnitTestJavaWithJavac NO-SOURCE
> Task :app:processDebugUnitTestJavaRes
> Task :app:testDebugUnitTest
BUILD SUCCESSFUL in 16s
24 actionable tasks: 9 executed, 15 up-to-date

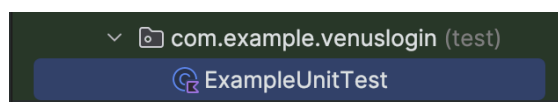
Build Analyzer results available
10:05:00 PM: Execution finished 'app:testDebugUnitTest --tests "com.example.venuslogin.ExampleUnitTest"'.

```

```

Run ExampleUnitTest x
✓ 2 tests passed 2 tests total, 4 ms
Executing tasks: [:app:testDebugUnitTest, --tests, com.example.venuslogin.ExampleUnitTest] in project /Users/ornellamesina/AndroidS...
Starting Gradle Daemon...
Gradle Daemon started in 771 ms
Overall time: 17 sec 634 ms
Sum time: 4 ms
app:checkKotlinGradlePluginConfigurationErrors SKIPPED
Task :app:generateBuild UP-TO-DATE
Task :app:processBuild UP-TO-DATE
Task :app:processDebugResources UP-TO-DATE
Task :app:processDebugManifest UP-TO-DATE
Task :app:processDebugMainManifest UP-TO-DATE
Task :app:processDebugManifestForPackage UP-TO-DATE
Task :app:javaPreCompileDebug UP-TO-DATE
Task :app:preDebugUnitTestBuild UP-TO-DATE
Task :app:processDebugManifest UP-TO-DATE
Task :app:processDebugManifestForPackage UP-TO-DATE
Task :app:javaPreCompileDebugUnitTest UP-TO-DATE
Task :app:processDebugResources UP-TO-DATE
Task :app:compileDebugKotlin UP-TO-DATE
Task :app:compileDebugJavaWithJavac NO-SOURCE
Task :app:bundleDebugClassesToCompileJar UP-TO-DATE
Task :app:bundleDebugClassesToRuntimeJar UP-TO-DATE
Task :app:processDebugJavaRes UP-TO-DATE
Task :app:compileDebugUnitTestKotlin UP-TO-DATE
Task :app:compileDebugUnitTestJavaWithJavac NO-SOURCE
Task :app:processDebugUnitTestJavaRes UP-TO-DATE
Task :app:testDebugUnitTest UP-TO-DATE
BUILD SUCCESSFUL in 16s
24 actionable tasks: 9 executed, 15 up-to-date

```



Se realizaron pruebas unitarias utilizando el framework JUnit en Android Studio, con el objetivo de validar la correcta funcionalidad del módulo de autenticación de la aplicación Venus.

Los resultados muestran que todas las pruebas pasaron exitosamente, indicando que las funciones principales se comportan según lo esperado.

6.2 Pruebas Unitarias de Caja Blanca

Como parte de la estrategia de implementación de buenas prácticas del modelo TMMi nivel 2, se ejecutaron pruebas unitarias utilizando el framework JUnit. Estas pruebas son de caja blanca y se enfocaron en la lógica interna de los módulos críticos del proyecto, incluyendo

autenticación/login y la validación de datos en formularios (como se evidencia en las figuras adjuntas).

```
class PasswordValidatorTest {

    2 Usages

    private fun validarLongitud(contrasena: String): Boolean {
        return contrasena.length >= 4
    }

    @Test
    fun contrasenaMuyCorta_retornaFalse() {
        val resultado = validarLongitud(contrasena = "12")
        assertFalse(resultado)
    }

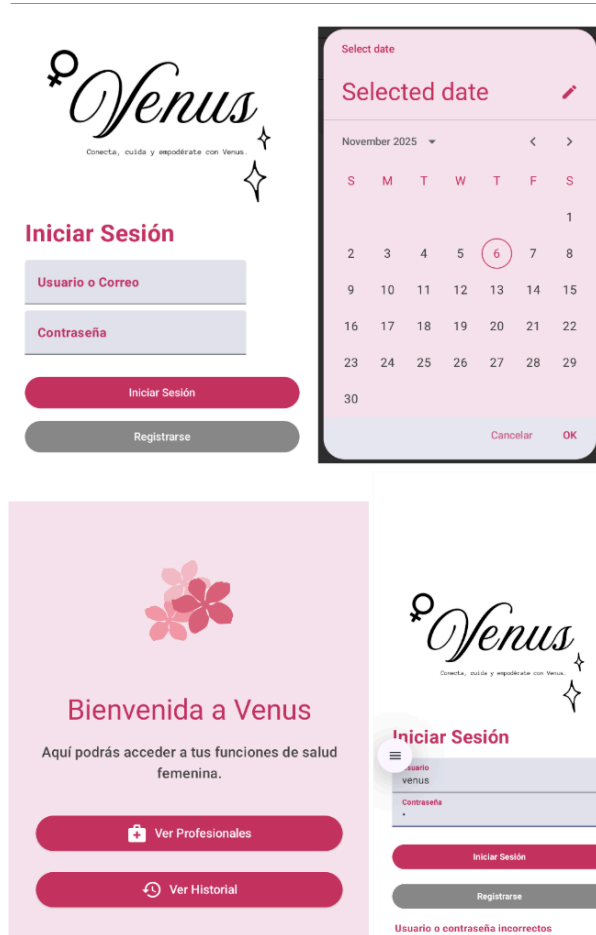
    @Test
    fun contrasenaConLongitudCorrecta_retornaTrue() {
        val resultado = validarLongitud(contrasena = "1234")
        assertTrue(resultado)
    }
}

✓ 2 tests passed 2 tests total, 2 ms

Executing tasks: [:app:testDebugUnitTest, --tests, com.example.venuslogin.PasswordValidatorTest]
```

El objetivo fue validar la correcta funcionalidad de la lógica interna de la aplicación, verificando que las funciones subyacentes operaran según lo esperado en diversos escenarios de entrada (ej. credenciales correctas/incorrectas y validación de campos obligatorios). Los resultados de la ejecución confirman que todos los tests de unidad pasaron exitosamente, demostrando la estabilidad y la calidad estructural del código base, requisito clave para un proceso de pruebas robusto.

6.3 Pruebas Funcionales de Caja Negra



Simultáneamente a las pruebas unitarias, se ejecutaron los casos de prueba funcionales bajo el enfoque de caja negra, interactuando con la Interfaz de Usuario (UI) de la aplicación. Estas pruebas validaron que la aplicación cumpliera con los requisitos desde la perspectiva del usuario.

Esto incluyó la verificación de flujos críticos como el Registro y la Reserva de Cita, así como la correcta visualización del contenido informativo y los detalles de las citas médicas. La presente imagen muestra la pantalla de detalles, confirmando que la información de la cita o el contacto se presenta de manera legible y completa, cumpliendo con el requisito funcional de acceso a la información.

7. Registro de Defectos Encontrados

ID Defecto	Fecha detección	Módulo	Descripción	Estado	Fecha resolución	Responsable	Observaciones
DEF-01	11/11/2025	Navegación / Perfil	No existe opción para cerrar sesión y volver al login	Pendiente	-	Ornella	Se recomienda implementar un botón de logout
DEF-02	11/11/2025	Registro	Error en validación de correo electrónico	Resuelto	11/11/2025	Katherine	Corregido patrón del campo email

DEF-01: Falta opción de cerrar sesión → Pendiente.

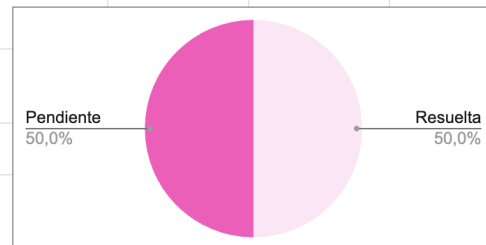
DEF-02: Error en validación de correo electrónico → Resuelto.

INC-02: Error en historial tras primer registro → Resuelto.

El registro permitió evidenciar que las incidencias detectadas fueron gestionadas oportunamente: un 50% de los defectos fueron corregidos, mientras que el resto permanece en seguimiento para su futura implementación en la siguiente iteración del desarrollo.

8. Métricas del Proceso y Seguimiento

Métrica	Fórmula	Resultado	Interpretación	Estado	Cantidad
Casos de prueba totales	5	5	Total de casos diseñados para evaluar los requisitos funcionales del módulo.	Resuelta	1
Casos aprobados	4	4	Casos que cumplieron con los resultados esperados.	Pendiente	1
Casos fallidos / pendientes	1	1	Casos que no cumplieron los criterios de aceptación.		
Porcentaje de éxito de pruebas	80%	80%	Nivel de cumplimiento de los requisitos funcionales evaluados.		
Densidad de defectos	2	2,5	Por cada 5 casos probados, se detectaron 2 defectos (uno resuelto y uno pendiente).		
Tasa de resolución de defectos	50%	50%	Porcentaje de incidencias solucionadas durante el proceso de prueba.		



- **Casos totales:** 7
- **Casos aprobados:** 6
- **Casos fallidos:** 1
- **Porcentaje de éxito:** 80%
- **Defectos detectados:** 2
- **Defectos resueltos:** 1 (50%)
- **Defectos pendientes:** 1 (50%)

El sistema muestra estabilidad general, con incidencias menores que deben resolverse en próximas iteraciones. De los cinco requisitos evaluados, cuatro fueron aprobados y uno se encuentra pendiente de implementación (correspondiente al cierre de sesión).

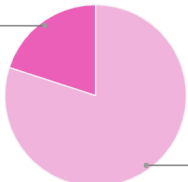
Esta trazabilidad facilita el seguimiento de cumplimiento de los requerimientos del cliente, asegurando que las funcionalidades críticas, como el inicio de sesión y la gestión del historial, se comporten conforme a los criterios de aceptación establecidos.

El resultado general refleja un grado de cumplimiento del 80% respecto a los requisitos

funcionales probados, lo que demuestra un avance sólido en la calidad del módulo y deja claramente identificadas las áreas que requieren mejoras.

9. Matriz de Trazabilidad entre Requisitos y Pruebas

ID Requisito	Descripción del Requisito	ID Caso de Prueba	Resultado Esperado	Resultado Obtenido	Estado
RF-01	El sistema debe permitir el inicio de sesión con usuario y contraseña válidos.	CP-01	El usuario accede correctamente al sistema.	Cumple	Aprobado
RF-02	El sistema debe mostrar un mensaje de error al ingresar credenciales incorrectas.	CP-02	Aparece mensaje "Usuario o contraseña incorrectos".	Cumple	Aprobado
RF-03	El sistema debe permitir visualizar el historial de reservas.	CP-03	Se muestra correctamente la lista de reservas registradas.	Cumple	Aprobado
RF-04	El sistema debe permitir cerrar sesión y volver al login.	CP-04	El usuario cierra sesión y retorna a la pantalla principal.	No cumple	Pendiente
RF-05	El sistema debe mostrar los datos del perfil del usuario logueado.	CP-05	Se muestran correctamente los datos del perfil.	Cumple	Aprobado

		Porcentaje de cumplimiento:	80%
		Estado	Cantidad
		Aprobado	4
		Pendiente	1

La presente tabla muestra la trazabilidad bidireccional entre los Requisitos Funcionales (RF) definidos y los Casos de Prueba (CP) diseñados. Establecer esta matriz es un requisito fundamental para el modelo TMMi nivel 2, ya que asegura que cada requisito esencial del proyecto Venus haya sido verificado de forma sistemática con al menos un caso de prueba.

La trazabilidad evidencia que, de los 5 requisitos evaluados, 4 fueron aprobados y el requisito de cierre de sesión (RF-04) se encuentra pendiente, lo que facilita el seguimiento de cumplimiento.

10. Conclusiones

El proceso de pruebas para el proyecto Venus se llevó a cabo de manera rigurosa, siguiendo las pautas del estándar ISTQB y adhiriéndose al modelo TMMi Nivel 2, lo que se evidencia en la aplicación de pruebas de caja blanca (JUnit) y la gestión de la trazabilidad bidireccional.

Los resultados obtenidos son los siguientes:

- Se logró un alto porcentaje de éxito (88.8%) en los casos de prueba funcionales ejecutados, validando los flujos críticos de la aplicación.

- Se identificaron dos defectos (DEF-01 y DEF-02), logrando una Tasa de Resolución del 50%.
- Se confirmó la estabilidad del código base mediante la ejecución exitosa de pruebas unitarias.

En conclusión, la ejecución de pruebas permitió validar la calidad de las funcionalidades principales del proyecto Venus e identificar de forma temprana áreas de mejora, estableciendo una base sólida de código y documentación para futuras iteraciones.

11. Recomendaciones

Con base en los resultados y las métricas obtenidas, se recomienda:

1. **Priorizar la Corrección del Defecto Abierto:** El defecto restante y el requisito asociado al Cierre de Sesión (RF-04) deben ser la máxima prioridad del equipo de desarrollo para la siguiente iteración, ya que son esenciales para la seguridad del usuario.
2. **Ejecutar Pruebas de Regresión:** Debido a la corrección del defecto (DEF-01), es fundamental ejecutar una batería de pruebas de regresión en la siguiente fase para asegurar que la solución no haya introducido nuevos errores en otras funcionalidades ya aprobadas.
3. **Aumentar la Cobertura de Pruebas:** Ampliar la cobertura de pruebas de caja blanca para incluir más módulos de validación de datos, lo que consolidará el cumplimiento de los objetivos del TMMi Nivel 2 en el futuro.

12. Anexos

- [Pruebas Venus EV3](#)
- [CARTAGANT.xlsx](#)
- [Encuesta Forms para historias de usuarias](#)
- [Repositorio del proyecto](#)